

*Numpy*模块的使用



A person in a white lab coat is working at a desk. They are using a laptop, a tablet, and looking at various papers and charts. The scene is partially obscured by a large blue arrow graphic pointing towards the right.

目录

- 01 数组的创建与操作
- 02 数组的基本运算方法
- 03 常用数学和统计函数
- 04 线性代数的求解
- 05 伪随机数的生成

数组的创建与操作

数组的创建的背景

```
# 身高
height = [176, 158, 163, 177, 172, 169]
# 体重
weight = [82, 61, 70, 69, 89, 78]
# BMI指数 (身体质量指数=体重(Kg)/身高(m)的平方)
BMI = weight / (height/100)**2
```

```
TypeError                                Traceback (most recent call last)
<ipython-input-1-4c0c58179eca> in <module>
      4 weight = [82, 61, 70, 69, 89, 78]
      5 # BMI指数 (身体质量指数=体重(Kg)/身高(m)的平方)
--> 6 BMI = weight / (height/100)**2

TypeError: unsupported operand type(s) for /: 'list' and 'int'
```

列表只是一种数据的存储容器，它不具有任何计算能力！

数组的创建与操作

数组的创建的背景

解决方案 (将两个列表中的对应元素取出来, 做数学运行)

```
BMI = []
```

```
for i in range(len(height)):
```

```
    BMI.append(weight[i] / (height[i]/100)**2)
```

```
BMI
```

```
[26.47210743801653,  
24.435186668803073,  
26.346494034400994,  
22.024322512687924,  
30.08382909680909,  
27.309968138370508]
```

有没有更好
的方法呢



数组的创建与操作



[User Guide](#) [API reference](#) [Development](#) [Release notes](#) [Learn](#)

Q Search the docs ...

NumPy documentation

Version: 1.24

Download documentation: [PDF Version](#) | [Historical versions of documentation](#)

Useful links: [Installation](#) | [Source Repository](#) | [Issue Tracker](#) | [Q&A Support](#) | [Mailing List](#)

NumPy is the fundamental package for scientific computing in Python. It is a Python library that provides a multidimensional array object, various derived objects (such as masked arrays and matrices), and an assortment of routines for fast operations on arrays, including mathematical, logical, shape manipulation, sorting, selecting, I/O, discrete Fourier transforms, basic linear algebra, basic statistical operations, random simulation and much more.



Getting Started

New to NumPy? Check out the Absolute Beginner's Guide. It contains an introduction to NumPy's main concepts and links to additional tutorials.

To the absolute beginner's
guide



User Guide

The user guide provides in-depth information on the key concepts of NumPy with useful background information and explanation.

To the user guide

数组的创建与操作

一维数组的创建

```
# 导入第三方模块
import numpy as np

# 将列表转换为数组
Height = np.array(height)
Weight = np.array(weight)
BMI = Weight / (Height/100)**2

# 返回BMI的值
BMI

array([26.47210744, 24.43518667, 26.34649403, 22.02432251, 30.0838291 ,
       27.30996814])
```

借助于array函数可以将列表或元组转换为数组

数组的创建与操作

二维数组的创建

基于嵌套列表创建二维数组

```
arr1 = np.array([[1,3,5,7],  
                 [2,4,6,8],  
                 [11,13,15,17],  
                 [12,14,16,18],  
                 [100,101,102,103]])
```

基于嵌套元组创建二维数组

```
arr2 = np.array(((8.5,6,4.1,2,0.7),(1.5,3,5.4,7.3,9),  
                 (3.2,3,3.8,3,3),(11.2,13.4,15.6,17.8,19)))
```

二维数组的打印结果

```
print(arr1, '\n')  
print(arr2)
```

out:

```
[[ 1  3  5  7]  
 [ 2  4  6  8]  
 [11 13 15 17]  
 [12 14 16 18]  
 [100 101 102 103]]
```

```
[[ 8.5  6.  4.1  2.  0.7]  
 [ 1.5  3.  5.4  7.3  9.]  
 [ 3.2  3.  3.8  3.  3.]  
 [11.2 13.4 15.6 17.8 19. ]]
```

数组的创建与操作

数组元素的返回

- 1) 在一维数组中，列表的所有索引方法都可以使用在数组中，而且还可以将任意位置的索引组装为列表，用作对应元素的获取；
- 2) 在二维数组中，位置索引必须写成[rows,cols]的形式，方括号的前半部分用于锁定二维数组的行索引，后半部分用于锁定数组的列索引；
- 3) 如果需要获取二维数组的所有行或列元素，那么，对应的行索引或列索引需要用英文状态的冒号表示；

数组的创建与操作

数组元素的返回

```
# 一维数组
arr1 = np.array([3,10,8,7,34,11,28,72])
# 一维数组元素的获取
print(arr1[[2,3,5,7]])

# 二维数组元素的获取
# 第2行第3列元素
print(arr2[1,2])
# 第3行所有元素
print(arr2[2,:])
# 第2列所有元素
print(arr2[:,1])
# 第2至4行，2至5列
print(arr2[1:4,1:5])
```

out:

[8 7 11 72]

5.4

[3.2 3. 3.8 3. 3.]

[6. 3. 3. 13.4]

[[3. 5.4 7.3 9.]

[3. 3.8 3. 3.]

[13.4 15.6 17.8 19.]]

数组的创建与操作

数组的常用属性

- ✦ `ndim` : 返回数组的维数 (返回1, 表示1维数组; 返回2, 表示二维数组)
- ✦ `shape` : 返回数组的行列数 (返回结果为元组结构)
- ✦ `dtype` : 返回数组元素的数据类型
- ✦ `size` : 返回数组元素的个数

数组的创建与操作

数组的常用属性

```
# 读入数据
stu_score = np.genfromtxt(fname = r'stu_socre.txt',
                           delimiter='\t', skip_header=1)

# 查看数据结构
print(type(stu_score))
# 查看数据维数
print(stu_score.ndim)
# 查看数据行列数
print(stu_score.shape)
# 查看数组元素的数据类型
print(stu_score.dtype)
# 查看数组元素个数
print(stu_score.size)
```

out:

```
<class 'numpy.ndarray'>
2
(1380, 5)
float64
6900
```

数组的创建与操作

数组的形状处理

- ✦ `reshape` : 用于转换数组的形状, 如原本 3×4 的数组转换成 6×2 的数组
- ✦ `resize` : 与`reshape`功能类似, 区别在于`reshape`无法原地修改数组的形状, 而`resize`可以
- ✦ `ravel` : 将多维数组转换为一维数组的方法
- ✦ `flatten` : 与`ravel`功能类似

数组的创建与操作

数组的形状处理

```
arr3 = np.array([[1,5,7],[3,6,1],[2,4,8],[5,8,9],[1,5,9],[8,5,2]])
```

```
# 数组的行列数
```

```
print(arr3.shape)
```

```
# 使用reshape方法更改数组的形状
```

```
print(arr3.reshape(2,9))
```

```
# 打印数组arr3的行列数
```

```
print(arr3.shape)
```

```
# 使用resize方法更改数组的形状，该方法无返回值
```

```
print(arr3.resize(2,9))
```

```
# 打印数组arr3的行列数
```

```
print(arr3.shape)
```

out:

(6, 3)

[[1 5 7 3 6 1 2 4 8]

[5 8 9 1 5 9 8 5 2]]

(6, 3)

None

(2, 9)

数组的创建与操作

数组的形状处理

```
# 构造3×3的二维矩阵
arr4 = np.array([[1,10,100],[2,20,200],[3,30,300]])
print('原数组 : \n',arr4)
# 默认排序降维 (默认排序按行的顺序降维)
print('数组降维 : \n',arr4.ravel())
print(arr4.flatten())
print(arr4.reshape(-1))
# 改变排序模式的降维 (按列的顺序降维)
print(arr4.ravel(order = 'F'))
print(arr4.flatten(order = 'F'))
print(arr4.reshape(-1, order = 'F'))
```

out:

原数组 :

```
[[ 1 10 100]
 [ 2 20 200]
 [ 3 30 300]]
```

数组降维 :

```
[ 1 10 100 2 20 200 3 30 300]
[ 1 10 100 2 20 200 3 30 300]
[ 1 10 100 2 20 200 3 30 300]
[ 1 2 3 10 20 30 100 200 300]
[ 1 2 3 10 20 30 100 200 300]
[ 1 2 3 10 20 30 100 200 300]
```

数组的创建与操作

数组的堆叠

- ✦ `row_stack` : 用于垂直方向 (纵向) 的数组堆叠
- ✦ `column_stack` : 用于水平方向 (横向) 的数组合并
- ✦ `vstack` : 其功能与`row_stack`函数一致
- ✦ `hstack` : 其功能与`column_stack`函数一致

数组的创建与操作

数组的堆叠

```
arr5 = np.array([1,2,3])
print('vstack纵向堆叠数组：',np.vstack([arr4,arr5]))
print('row_stack纵向堆叠数组：',np.row_stack([arr4,arr5]))
arr6 = np.array([[5],[15],[25]])
print('hstack横向合并数组：',np.hstack([arr4,arr6]))
print('column_stack横向合并数组：',np.column_stack([arr4,arr6]))
```

out:

vstack纵向堆叠数组：	row_stack纵向堆叠数组：
[[1 10 100]	[[1 10 100]
[2 20 200]	[2 20 200]
[3 30 300]	[3 30 300]
[1 2 3]]	[1 2 3]]

out:

hstack横向合并数组：

```
[[ 1 10 100 5]
 [ 2 20 200 15]
 [ 3 30 300 25]]
```

column_stack横向合并数组：

```
[[ 1 10 100 5]
 [ 2 20 200 15]
 [ 3 30 300 25]]
```


数组的基本运算方法

数学运算符

运算符	含义	运算符	含义
+	数组对应元素的加和	-	数组对应元素的差
*	数组对应元素的乘积	/	数组对应元素的商
%	数组对应元素商的余数	//	数组对应元素商的整除数
**	数组对应元素的幂指数		

数组的创建与操作

数学运算符

```
# 加法运算
math = np.array([98,83,86,92,67,82])
english = np.array([68,74,66,82,75,89])
chinese = np.array([92,83,76,85,87,77])
tot_symbol = math+english+chinese
tot_fun = np.add(np.add(math,english),chinese)
print('符号加法 : \n',tot_symbol)
print('函数加法 : \n',tot_fun)

# 除法运算
height = np.array([165,177,158,169,173])
weight = np.array([62,73,59,72,80])
BMI_symbol = weight/(height/100)**2
BMI_fun = np.divide(weight,np.divide(height,100)**2)
print('符号除法 : \n',BMI_symbol)
print('函数除法 : \n',BMI_fun)
```

out:

```
符号加法 :
[258 240 228 259 229 248]
函数加法 :
[258 240 228 259 229 248]
符号除法 :
[ 22.77318641  23.30109483
 23.63403301  25.20920136
 26.7299275 ]
函数除法 :
[ 22.77318641  23.30109483
 23.63403301  25.20920136
 26.7299275 ]
```

数组的创建与操作

数学运算符

```
arr7 = np.array([[1,2,10],[10,8,3],[7,6,5]])
arr8 = np.array([[2,2,2],[3,3,3],[4,4,4]])
print('数组arr7 : \n',arr7)
print('数组arr8 : \n',arr8)

# 求余数
print('计算余数 : \n',arr7 % arr8)

# 求整除
print('计算整除 : \n',arr7 // arr8)

# 求指数
print('计算指数 : \n',arr7 ** arr8)
```

out:

数组arr7 :	计算整除 :
[[1 2 10]	[[0 1 5]
[10 8 3]	[3 2 1]
[7 6 5]]	[1 1 1]]
数组arr8 :	计算指数 :
[[2 2 2]	[[1 4 100]
[3 3 3]	[1000 512 27]
[4 4 4]]	[2401 1296 625]]
计算余数 :	
[[1 0 0]	
[1 2 0]	
[3 2 1]]	

数组的基本运算方法

比较运算符

符号	函数	含义
>	<code>np.greater(arr1,arr2)</code>	判断arr1的元素是否大于arr2的元素
>=	<code>np.greater_equal(arr1,arr2)</code>	判断arr1的元素是否大于等于arr2的元素
<	<code>np.less(arr1,arr2)</code>	判断arr1的元素是否小于arr2的元素
<=	<code>np.less_equal(arr1,arr2)</code>	判断arr1的元素是否小于等于arr2的元素
==	<code>np.equal(arr1,arr2)</code>	判断arr1的元素是否等于arr2的元素
!=	<code>np.not_equal(arr1,arr2)</code>	判断arr1的元素是否不等于arr2的元素

数组的创建与操作

比较运算符

```
# 取子集
# 从arr7中取出arr7大于arr8的所有元素
print(arr7)
print('满足条件的二维数组元素获取：\n',arr7[arr7>arr8])
# 从arr9中取出大于10的元素
arr9 = np.array([3,10,23,7,16,9,17,22,4,8,15])
print('满足条件的一维数组元素获取：\n',arr9[arr9>10])

# 判断操作
# 将arr7中大于7的元素改成5，其余的不变
print('二维数组的条件操作：\n',np.where(arr7>7,5,arr7))
# 将arr9中大于10 的元素改为1，否则改为0
print('一维数组的条件操作：\n',np.where(arr9>10,1,0))
```

out:

```
[[ 1  2 10]      [[2 2 2]
 [10  8  3]      [3 3 3]
 [ 7  6  5]]     [4 4 4]]
[[False False  True]
 [ True  True False]
 [ True  True  True]]
满足条件的二维数组元素获取：
[10 10  8  7  6  5]
满足条件的一维数组元素获取：
[23 16 17 22 15]
二维数组的条件操作：
[[1 2 5]
 [5 5 3]
 [7 6 5]]
一维数组的条件操作：
[0 0 1 ..., 0 0 1]
```

常用的数学和统计函数

常用的数学函数

函数	函数说明
np.pi	常数pi
np.e	常数e
np.fabs(arr)	计算各元素的浮点型绝对值
np.ceil(arr)	对各元素向上取整
np.floor(arr)	对各元素向下取整
np.round(arr)	对各元素四舍五入
np.fmod(arr1,arr2)	计算arr1/arr2的余数

常用的数学和统计函数

常用的数学函数

函数	函数说明
<code>np.modf(arr)</code>	返回数组元素的小数部分和整数部分
<code>np.sqrt(arr)</code>	计算各元素的算术平方根
<code>np.square(arr)</code>	计算各元素的平方值
<code>np.exp(arr)</code>	计算以e为底的指数
<code>np.power(arr, α)</code>	计算各元素的指数
<code>np.log2(arr)</code>	计算以2为底各元素的对数
<code>np.log10(arr)</code>	计算以10为底各元素的对数
<code>np.log(arr)</code>	计算以e为底各元素的对数

常用的数学和统计函数

常用的统计函数

函数	函数说明
<code>np.min(arr,axis)</code>	按照轴的方向计算最小值
<code>np.max(arr,axis)</code>	按照轴的方向计算最大值
<code>np.mean(arr,axis)</code>	按照轴的方向计算平均值
<code>np.median(arr,axis)</code>	按照轴的方向计算中位数
<code>np.sum(arr,axis)</code>	按照轴的方向计算和
<code>np.std(arr,axis)</code>	按照轴的方向计算标准差
<code>np.var(arr,axis)</code>	按照轴的方向计算方差

常用的数学和统计函数

常用的统计函数

函数	函数说明
<code>np.cumsum(arr,axis)</code>	按照轴的方向计算累计和
<code>np.cumprod(arr,axis)</code>	按照轴的方向计算累计乘积
<code>np.argmin(arr,axis)</code>	按照轴的方向返回最小值所在的位置
<code>np.argmax(arr,axis)</code>	按照轴的方向返回最大值所在的位置
<code>np.corrcoef(arr)</code>	计算皮尔逊相关系数
<code>np.cov(arr)</code>	计算协方差矩阵

注意：axis=0时，计算数组各列的统计值；
axis=1时，计算数组各行的统计值

常用的数学和统计函数

常用的统计函数

```
print(arr4)
print('垂直方向计算数组的和 : \n',np.sum(arr4,axis = 0))
print('水平方向计算数组的和 : \n',np.sum(arr4, axis = 1))
```

out:

```
[[ 1 1000 3000]
 [ 2  20  200]
 [ 3  30  300]]
```

垂直方向 (列) 计算数组的和 :

```
[6 1050 3500]
```

水平方向 (行) 计算数组的和 :

```
[4001  222  333]
```

线性代数的求解

常用的线代函数（了解）

函数	说明	函数	说明
<code>np.zeros</code>	生成零矩阵	<code>np.ones</code>	生成所有元素为1的矩阵
<code>np.eye</code>	生成单位矩阵	<code>np.transpose</code>	矩阵转置
<code>np.dot</code>	计算两个数组的点积	<code>np.inner</code>	计算两个数组的内积
<code>np.diag</code>	矩阵主对角线与一维数组间的转换	<code>np.trace</code>	矩阵主对角线元素的和
<code>np.linalg.det</code>	计算矩阵行列式	<code>np.linalg.eig</code>	计算矩阵特征根与特征向量
<code>np.linalg.eigvals</code>	计算方阵特征根	<code>np.linalg.inv</code>	计算方阵的逆
<code>np.linalg.pinv</code>	计算方阵的Moore-Penrose伪逆	<code>np.linalg.solve</code>	计算 $Ax=b$ 的线性方程组的解
<code>np.linalg.lstsq</code>	计算 $Ax=b$ 的最小二乘解	<code>np.linalg.qr</code>	计算QR分解
<code>np.linalg.svd</code>	计算奇异值分解	<code>np.linalg.norm</code>	计算向量或矩阵的范数

线性代数的求解

常用的线代函数（了解）

```
# 计算方阵的特征向量和特征根
arr16 = np.array([[1,2,5],[3,6,8],[4,7,9]])
print('计算3×3方阵的特征根和特征向量：\n',arr16)
print('求解结果为：\n',np.linalg.eig(arr16))
```

out:

计算3×3方阵的特征根和特征向量：

```
[[1 2 5]
 [3 6 8]
 [4 7 9]]
```

求解结果为：

```
(array([ 16.75112093, -1.12317544,  0.37205451]),
 array([[ -0.30758888, -0.90292521,  0.76324346],
        [-0.62178217, -0.09138877, -0.62723398],
        [-0.72026108,  0.41996923,  0.15503853]]))
```

线性代数的求解

常用的线代函数（了解）

```
# 多元线性方程组
```

```
A = np.array([[3,2,1],[2,3,1],[1,2,3]])
```

```
b = np.array([39,34,26])
```

```
X = np.linalg.solve(A,b)
```

```
print('三元一次方程组的解：\n',X)
```

```
out:
```

```
三元一次方程组的解：
```

```
[ 9.25  4.25  2.75]
```

$$\begin{cases} 3x + 2y + z = 39 \\ 2x + 3y + z = 34 \\ x + 2y + 3z = 26 \end{cases}$$

线性代数的求解

常用的线代函数 (了解)

```
# 计算偏回归系数
```

```
X = np.array([[1,1,4,3],[1,2,7,6],[1,2,6,6],[1,3,8,7],[1,2,5,8],  
[1,3,7,5],[1,6,10,12],[1,5,7,7],[1,6,3,4],[1,5,7,8]])
```

```
Y = np.array([3.2,3.8,3.7,4.3,4.4,5.2,6.7,4.8,4.2,5.1])
```

```
X_trans_X_inverse = np.linalg.inv(np.dot(np.transpose(X),X))
```

```
beta = np.dot(np.dot(X_trans_X_inverse,np.transpose(X)),Y)
```

```
print('偏回归系数为 : \n',beta)
```

```
print('原始的Y= : \n', Y)
```

```
print('模型估计的Y=: \n', np.dot(X, beta))
```

out:

偏回归系数为 :

```
[ 1.78052227  0.24720413  
 0.15841148  0.13339845]
```

原始的Y= :

```
[ 3.2  3.8  3.7  4.3  4.4  5.2  6.7  
 4.8  4.2  5.1]
```

模型估计的Y=:

```
[ 3.06156768  4.18420162  
 4.02579014  4.72321568  
 4.13417557  4.29800729  
 6.44864331  5.05921247  
 4.27257532  5.19261092]
```

伪随机数的生成

常用的随机分布函数（了解）

函数	说明
<code>seed(n)</code>	设置随机种子
<code>beta(a, b, size=None)</code>	生成贝塔分布随机数
<code>chisquare(df, size=None)</code>	生成卡方分布随机数
<code>choice(a, size=None, replace=True, p=None)</code>	从a中有放回地随机挑选指定数量的样本
<code>exponential(scale=1.0, size=None)</code>	生成指数分布随机数
<code>f(dfnum, dfden, size=None)</code>	生成F分布随机数
<code>gamma(shape, scale=1.0, size=None)</code>	生成伽马分布随机数
<code>geometric(p, size=None)</code>	生成几何分布随机数
<code>hypergeometric(ngood, nbad, nsample, size=None)</code>	生成超几何分布随机数
<code>laplace(loc=0.0, scale=1.0, size=None)</code>	生成拉普拉斯分布随机数
<code>logistic(loc=0.0, scale=1.0, size=None)</code>	生成Logistic分布随机数
<code>lognormal(mean=0.0, sigma=1.0, size=None)</code>	生成对数正态分布随机数

伪随机数的生成

常用的随机分布函数（了解）

函数	说明
negative_binomial(n, p, size=None)	生成负二项分布随机数
multinomial(n, pvals, size=None)	生成多项分布随机数
multivariate_normal(mean, cov[, size])	生成多元正态分布随机数
normal(loc=0.0, scale=1.0, size=None)	生成正态分布随机数
pareto(a, size=None)	生成帕累托分布随机数
poisson(lam=1.0, size=None)	生成泊松分布随机数
rand(d0, d1, ..., dn)	生成n维的均匀分布随机数
randn(d0, d1, ..., dn)	生成n维的标准正态分布随机数
randint(low, high=None, size=None, dtype='l')	生成指定范围的随机整数
random_sample(size=None)	生成[0,1)的随机数
standard_t(df, size=None)	生成标准的t分布随机数
uniform(low=0.0, high=1.0, size=None)	生成指定范围的均匀分布随机数
wald(mean, scale, size=None)	生成Wald分布随机数
weibull(a, size=None)	生成Weibull分布随机数

伪随机数的生成

常用的随机分布函数（了解）

```
# 导入第三方模块
import seaborn as sns
import matplotlib.pyplot as plt
from scipy import stats

# 生成各种正态分布随机数
np.random.seed(1234)
rn1 = np.random.normal(loc = 0, scale = 1, size = 1000)
rn2 = np.random.normal(loc = 0, scale = 2, size = 1000)
rn3 = np.random.normal(loc = 2, scale = 3, size = 1000)
rn4 = np.random.normal(loc = 5, scale = 3, size = 1000)

# 绘图
plt.style.use('ggplot')
sns.distplot(rn1, hist = False, kde = False, fit = stats.norm,
             fit_kws = {'color':'black','label':'u=0,s=1','linestyle':'-'})
```

伪随机数的生成

常用的随机分布函数（了解）

续上一页的代码

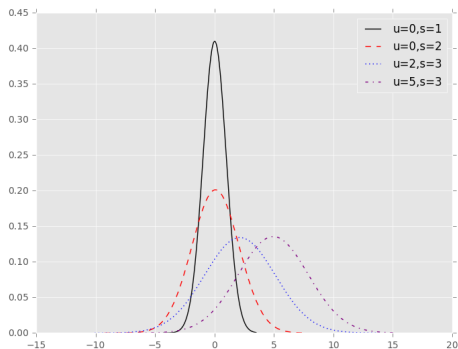
```
sns.distplot(rn2, hist = False, kde = False, fit = stats.norm,  
             fit_kws = {'color':'red','label':'u=0,s=2','linestyle':'--'})  
sns.distplot(rn3, hist = False, kde = False, fit = stats.norm,  
             fit_kws = {'color':'blue','label':'u=2,s=3','linestyle':'.'})  
sns.distplot(rn4, hist = False, kde = False, fit = stats.norm,  
             fit_kws = {'color':'purple','label':'u=5,s=3','linestyle':'-.'})
```

呈现图例

plt.legend()

呈现图形

plt.show()



伪随机数的生成

常用的随机分布函数（了解）

```
# 生成各种指数分布随机数
np.random.seed(1234)
re1 = np.random.exponential(scale = 0.5, size = 1000)
re2 = np.random.exponential(scale = 1, size = 1000)
re3 = np.random.exponential(scale = 1.5, size = 1000)
# 绘图
sns.distplot(re1, hist = False, kde = False, fit = stats.expon,
              fit_kws = {'color':'black','label':'lambda=0.5','linestyle':'-'})
sns.distplot(re2, hist = False, kde = False, fit = stats.expon,
              fit_kws = {'color':'red','label':'lambda=1','linestyle':'--'})
sns.distplot(re3, hist = False, kde = False, fit = stats.expon,
              fit_kws = {'color':'blue','label':'lambda=1.5','linestyle':'.'})
# 呈现图例
plt.legend()
# 呈现图形
plt.show()
```

